智能在线编程练习平台 软件详细设计文档（SDD）

1 引言

1.1 编写目的

本文档基于**需求分析文档（SRS）** 完成，对智能在线编程练习平台进行全维度软件详细设计，明确系统架构、模块划分、数据库设计、接口设计、部署方案、安全与性能规范，为后端/前端开发、测试用例设计、系统部署提供**唯一执行依据**，保障系统按需求落地、稳定交付。

1.2 项目背景

传统算法刷题平台存在**功能单一、判题僵化、无智能指导、个性化不足**等痛点，本项目融合**自研 C++ OJ 判题引擎**与 **AI 大模型**，基于 SpringBoot 开发 Web 端轻量化算法学习平台，实现「AI 出题→在线做题→自动判题→AI 评题→优化改进」全流程闭环，服务学生、求职者、竞赛选手等算法学习人群。

1.3 术语定义

术语	全称/解释
OJ	Online Judge，在线判题系统
SDD	Software Design Document，软件详细设计文档
JWT	JSON Web Token，用户身份认证令牌
沙箱	代码隔离运行环境，防止恶意代码攻击
双引擎	传统 OJ 判题引擎 + AI 大模型智能引擎
BCrypt	密码加密算法

---

2 总体设计

2.1 系统架构

采用**前后端分离 + 双引擎核心**架构，分层设计保障可扩展性与稳定性：

1. **前端层**：在线代码编辑器、页面交互、数据可视化
2. **后端服务层**：SpringBoot 业务逻辑、接口提供、模块调度
3. **核心引擎层**：C++ OJ 判题引擎（准确性）、AI 大模型引擎（智能化）
4. **数据层**：MySQL 数据库、文件存储（测试用例/头像）
5. **安全层**：沙箱隔离、权限校验、数据加密

2.2 模块划分

系统划分为 **7 大核心模块**，模块间低耦合、高内聚：

1. 用户管理模块
2. 题库管理模块
3. 在线编程与评测模块
4. AI 智能辅助模块
5. 数据统计模块
6. 后台管理模块
7. 公共工具模块

2.3 技术选型

层级	技术栈
后端	SpringBoot 、JDK 17、Maven
数据库	MySQL 、utf8mb4 字符集
判题引擎	C++ GCC、安全沙箱组件
AI 对接	火山引擎-豆包大模型平台
前端	VS Code、在线代码编辑器、数据可视化组件

协作工具	Github

2.4 运行环境

- **开发环境**：Windows10+、i7+CPU、12G+内存、IDEA/VS Code
- **测试环境**：Chrome/Edge 主流浏览器
- **生产环境**：隔离沙箱、MySQL

---

3 模块详细设计

3.1 用户管理模块

3.1.1 核心功能

账号注册/登录、个人信息管理、密码修改、提交记录查询、刷题数据统计、Token 认证

3.1.2 核心流程

1. 注册：用户名/邮箱校验→密码加密→邮箱验证→创建用户
2. 登录：账号密码校验→生成 JWT Token→返回用户信息
3. 个人中心：信息修改/头像上传→数据更新→同步展示

3.1.3 类设计

类名	类型	核心职责
UserController	控制层	接收用户请求、参数校验、响应返回
UserService	业务层	注册/登录/信息管理业务逻辑
UserMapper	数据层	用户表 CRUD
User	实体类	用户数据封装
JwtUtil	工具类	Token 生成/校验/过期处理

3.2 题库管理模块

3.2.1 核心功能

题目分类管理、题目增删改查、测试用例管理、官方题解管理、题目检索筛选

3.2.2 核心流程

1. 题目管理：新增/编辑→校验参数→存储题目/测试用例→上架展示
2. 分类管理：新增/删除→关联题目→前端展示筛选
3. 题解管理：编辑/发布→审核→前端展示

3.2.3 类设计

类名	类型	核心职责
ProblemController	控制层	题目/分类/题解接口
ProblemService	业务层	题库业务逻辑
ProblemMapper	数据层	题目/分类/关联表 CRUD
Category	实体类	题目分类数据
Problem	实体类	题目数据

3.3 在线编程与评测模块

3.3.1 核心功能

在线代码编辑、代码提交、编译运行、自动判分、沙箱隔离、结果返回

3.3.2 核心流程

1. 代码提交：参数校验→恶意代码检测→加入判题队列
2. 自动判分：创建沙箱→编译→隔离运行→测试用例比对→结果汇总
3. 结果返回：推送判分结果→存储提交记录→清理沙箱

3.3.3 类设计

类名	类型	核心职责
JudgeController	控制层	代码提交、判分结果查询
JudgeService	业务层	判题调度、沙箱调用
SubmissionMapper	数据层	提交记录 CRUD
SandboxUtil	工具类	沙箱环境创建、资源限制
OJEngine	引擎类	C++编译、运行、用例比对

3.4 AI 智能辅助模块

3.4.1 核心功能

AI 个性化出题、代码多维评价、智能题解生成、编程答疑

3.4.2 核心流程

1. AI 出题：参数输入→调用大模型→生成题目→存储→展示
2. AI 评题：提交记录→调用模型→复杂度/风格/优化分析→存储评价
3. AI 答疑：用户提问→上下文匹配→模型解答→返回结果

3.4.3 类设计

类名	类型	核心职责
AIController	控制层	AI 功能接口
AIService	业务层	AI 接口调用、结果解析
EvaluationMapper	数据层	AI 评价 CRUD
AIUtil	工具类	大模型接口对接、限流控制

3.5 数据统计模块

3.5.1 核心功能

个人刷题统计、学习数据可视化、平台运营数据统计

3.5.2 类设计

类名	类型	核心职责
StatisticsController	控制层	统计数据接口
StatisticsService	业务层	数据计算、可视化封装
UserStatisticsMapper	数据层	用户做题统计 CRUD

3.6 后台管理模块

3.6.1 核心功能

用户管理、题库管理、AI 内容审核、运营监控、权限控制

3.6.2 核心流程

管理员登录→权限校验→后台操作→数据审核→结果同步

3.6.3 类设计

类名	核心职责
AdminController	管理员接口、权限拦截
AdminService	后台管理业务逻辑
PermissionInterceptor	拦截器

3.7 公共工具模块

核心工具：全局异常处理、日志记录、文件上传、密码加密、请求限流、参数校验

工具类：GlobalExceptionHandler、LogUtil、FileUploadUtil、BCryptUtil

4 数据库详细设计

4.1 数据库基础信息

- 数据库名：`oj_platform`
- 字符集：utf8mb4
- 排序规则：utf8mb4_unicode_ci
- 存储引擎：InnoDB

4.2 核心表结构（精简版）

1. **用户表（user）**：用户 ID、用户名、加密密码、邮箱、角色、头像
2. **题目分类表（category）**：分类 ID、分类名
3. **题目表（problem）**：题目 ID、标题、难度、描述、时间/内存限制
4. **题目-分类关联表（problem_category）**：多对多关联
5. **提交记录表（submission）**：提交 ID、用户 ID、题目 ID、代码、判分结果
6. **AI 评价表（evaluation）**：评价 ID、提交 ID、复杂度、优化建议
7. **题解表（solution）**：题解 ID、题目 ID、内容、代码
8. **用户 Token 表（user_token）**：Token、用户 ID、过期时间
9. **用户做题统计表（user_statistics）**：总提交、总通过
10. **用户题目状态表（user_problem）**：用户题目完成状态
11. **题目测试样例表（problem_test_case）**：输入/输出文件 URL

4.3 索引与约束

- 唯一索引：用户名、邮箱、手机号、用户-题目联合索引
- 外键约束：提交记录→用户/题目、AI 评价→提交记录
- 非空约束：核心业务字段强制非空

4.4 E-R 核心关系

- 用户 ←→ 提交记录：一对多
- 题目 ←→ 提交记录：一对多

- 题目 ←→ 分类：多对多
- 提交记录 ←→ AI 评价：一对一
- 题目 ←→ 题解：一对多

---

5 接口详细设计

5.1 接口规范

- 基础路径：`/api/v1`
- 请求方式：GET/POST/PUT/DELETE
- 数据格式：JSON
- 认证方式：请求头携带 `Bearer {Token}`
- 状态码：200 成功、400 参数错误、401 未登录、403 无权限、500 服务异常

5.2 核心接口分类

1. **认证接口**：注册、登录、退出、密码重置
2. **用户接口**：信息管理、提交记录、刷题统计
3. **题库接口**：分类列表、题目列表、题目详情、题解
4. **评测接口**：代码提交、判分结果查询
5. **AI 接口**：AI 出题、代码评价、智能题解
6. **管理员接口**：用户管理、题目管理、分类管理、测试用例管理

5.3 异常处理

- 全局异常处理器捕获所有异常
- 自定义业务异常（用户已存在、题目不存在、判分失败等）
- 统一响应格式：`{"code":xxx,"message":"xxx","data":xxx}`

---

6 部署与环境设计

6.1 部署架构

单机部署模式： MySQL + 沙箱环境 + AI 接口

6.2 部署步骤

1. 环境搭建：JDK、MySQL、GCC、沙箱组件
2. 数据库初始化：执行 SQL 脚本创建库表
3. 代码部署：打包 SpringBoot 项目、上传服务器
4. 引擎配置：OJ 引擎编译、沙箱权限配置
5. AI 对接：配置大模型 API 密钥、限流规则
6. 启动验证：接口测试、功能联调

6.3 沙箱安全配置

- 代码隔离运行，单次运行后销毁
- 限制 CPU/内存/运行时间
- 禁止网络访问、危险系统调用
- 恶意代码检测拦截

---

7 安全设计

7.1 数据安全

- 密码 BCrypt 加密存储，无明文
- 敏感数据传输加密
- 定期数据备份

7.2 接口安全

- JWT 认证，过期自动失效
- 管理员权限拦截，越权操作 100%拦截
- 请求限流：单用户每秒最多提交 1 次

7.3 运行安全

- 沙箱隔离，杜绝恶意代码

- 代码提交前恶意代码检测

7.4 内容安全

- AI 生成内容审核
- 违规内容自动过滤
- 符合国家法律法规要求

---

8 性能设计

8.1 性能指标（验收标准）

- 页面首屏加载≤1.5s，切换≤0.5s
- 常规查询≤300ms，写入≤500ms
- 常规题目判分≤8s，平均≤3s
- 支持 1000 用户在线，200 并发

8.2 优化方案

- 数据库索引优化，避免全表扫描
- 异步处理判题、AI 生成任务
- 接口结果缓存，减少重复计算
- 代码规范评审，提升执行效率

---

9 测试设计

9.1 测试策略

- 单元测试：核心业务方法、工具类
- 集成测试：模块联调、接口调用
- 系统测试：全功能流程、性能、安全
- 验收测试：按 SRS 需求验证

9.2 核心测试点

1. 功能：注册登录、题目刷题、AI 功能、后台管理
2. 判题：准确性 100%、无错判漏判
3. 安全：越权、恶意代码、数据泄露
4. 性能：并发、响应时间、判分速度

---

10 附录

10.1 缩略语

- OJ：在线判题
- AI：人工智能
- JWT：身份令牌
- SpringBoot：后端开发框架
- GCC：C++编译器